Kenbright 360

Complete Project Development Plan

by

Robin Ochieng
Meshack Wanjaria

Project Philosophy: The MVP Agile Approach

We will build Kenbright 360 using an Agile Methodology, focusing on a Minimum Viable Product (MVP) for Version 1.0. This means we prioritize the most critical features that deliver the core value of the loyalty program. This approach allows for:

- **Early Testing:** Get a working product into the hands of users and staff quickly.
- **Faster Launch:** Reduce time-to-market for the core program.
- **Reduced Risk:** Avoid building complex, unnecessary features upfront.
- **Iterative Improvement:** Gather feedback and add more advanced features in subsequent versions.

Phase 1: The Foundation - Database & Backend API

- This phase is about building the brain and central nervous system of the application. The frontend will be a separate layer that communicates with this backend.

Step 1.1: Comprehensive Database Schema Design

- Purpose: To create a robust, scalable, and logically structured database that serves as the single source of truth for all loyalty program operations. A poor design here will cause critical failures later.

Core Tables & Their Detailed Structure:

1. members Table (The Central Identity Hub)

- Why: Every individual in the system must be uniquely identifiable and their core profile stored here. This is the starting point for all relationships.

Fields:

- member_id (Primary Key, Auto-increment)
- national_id (Unique, for KYC)
- phone_number (Unique, for login & communication)
- email_address
- first_name, last_name
- tier (ENUM: 'Bronze', 'Silver', 'Gold', 'Platinum') - Default: 'Bronze'
- referral_code (VARCHAR, Unique) - *e.g., 'KB-JANE-A1B2'*
- referred_by_member_id (Foreign Key to members.member_id, NULLable) - Tracks the referrer.
- created_at (Timestamp)

2. products Table (The Insurance Product Catalog)

Why: To define all Kenbright products and their points rules independently. This makes it easy to add, modify, or retire products without changing application code.

Fields:

- product_id (Primary Key, Auto-increment)
- product_name (e.g., 'Motor Insurance', 'Medical Cover')
- product_category (ENUM: 'Core', 'Optional')
- points_calculation_ratio (INT) - *The denominator for the points formula. e.g., 100 for 1 pt/100 KES, 1000 for 1 pt/1000 KES.*

3. member_products Table (The Member-Product Relationship)

Why: A member can have multiple products, and a product can be owned by many members. This "junction table" creates this many-to-many relationship. It is critical for tracking the "Products/Client" metric and for triggering tier upgrades.

Fields:

- id (Primary Key, Auto-increment)
- member_id (Foreign Key to members)
- product_id (Foreign Key to products)
- policy_number (VARCHAR)
- status (ENUM: 'Active', 'Expired', 'Lapsed') - Default: 'Active'
- unique_reference (VARCHAR) - e.g., Vehicle Registration Number for Motor Insurance.

4. payments Table (The Financial Transaction Ledger)

Why: To log every payment attempt for reconciliation, auditing, and triggering the points engine. This fulfills the "Receipt & Invoice Tracking" requirement.

Fields:

- payment_id (Primary Key, Auto-increment)
- member_id (Foreign Key, NULLable initially for suspense)
- product_id (Foreign Key, NULLable initially for suspense)
- amount_paid (DECIMAL)
- payment_date (Timestamp)
- paybill_number (VARCHAR) - The product-specific paybill used.
- unique_reference_used (VARCHAR) - The reference sent with the payment.
- status (ENUM: 'Matched', 'Suspense') - Default: 'Suspense'
- partner_code (VARCHAR, NULLable) - For "Identification of clients with partners".

5. loyalty_ledger Table (The Sacred Points Ledger)

Why: This is an append-only record of every single points transaction. It is the absolute source of truth for a member's points balance, ensuring accuracy and auditability.

Fields:

- ledger_id (Primary Key, Auto-increment)
- member_id (Foreign Key)
- payment_id (Foreign Key, NULLable if points are from a referral)
- points_earned (INT, Default: 0)
- points_redeemed (INT, Default: 0)
- cumulative_balance (INT) - Calculated as the running total.
- transaction_type (VARCHAR) - e.g., 'Purchase', 'Referral Bonus', 'Redemption'
- transaction_date (Timestamp)

6. rewards_catalog Table (The Points Store Inventory)

Why: To define the "Incentives & Discounts" that members can purchase with their points, enabling the "Points Usage (deleting)" functionality.

Fields:

- reward_id (Primary Key, Auto-increment)
- reward_name (e.g., '15% Off Home Insurance Renewal')
- points_cost (INT)
- description (TEXT)
- is_active (BOOLEAN, Default: TRUE)

7. points_redemptions Table (The Redemption Fulfillment Log)

Why: To track the lifecycle of a redemption request, from initiation by the user to fulfillment by the admin team.

Fields:

- redemption_id (Primary Key, Auto-increment)
- member_id (Foreign Key)
- reward_id (Foreign Key)
- points_spent (INT)
- redemption_date (Timestamp)
- status (ENUM: 'Requested', 'Processing', 'Fulfilled', 'Cancelled')

8. assets Table (For Cross-Selling - Physical Assets)

Why: To store information about a member's insurable assets, enabling personalized cross-selling (e.g., "You have a car, would you like a quote for your home?").

Fields:

- asset_id (Primary Key, Auto-increment)
- member_id (Foreign Key)
- asset_type (ENUM: 'Vehicle', 'Home', 'Personal Office')
- details (JSON) - Flexible field to store specific info like make, model, registration, value, location, etc.

9. dependents Table (For Cross-Selling - Family)

Why: To store information about a member's family for products like Medical insurance.

Fields:

- dependent_id (Primary Key, Auto-increment)
- member_id (Foreign Key)
- full_name
- relationship (ENUM: 'Spouse', 'Child', 'Parent', 'Sibling')
- date_of_birth (Date)

Step 1.2: Backend API & Calculation Engine Development

Purpose: To create a secure and reliable set of programming interfaces (APIs) that contain all the business logic. The admin dashboard and user portal will call these APIs to perform all actions.

Core API Endpoints & Engine Functions:

1. Authentication Endpoints

- POST /api/auth/login (For users, using phone/OTP or email/password)
- POST /api/auth/admin/login (For admins, using email/password)
- POST /api/auth/logout

Why: Secure access control for both user and admin systems.

2. Member Registration & KYC Endpoint

- POST /api/members

Logic:

- Validates incoming KYC data (National ID, Phone, etc.).
- Generates a unique referral_code for the new member.
- If a referral_code is provided during signup, it validates it and sets the referred_by_member_id.
- Creates the member with a default 'Bronze' tier.

Why: This is the digital version of the sales team's "First Purchase: Create Member Record" step.

3. Payment Reconciliation Engine (A Background Process)

Logic:

- Runs every few minutes, fetching payments with status='Suspense'.
- **Auto-Match:** Tries to find a member by looking up the unique_reference_used in the member_products table.
- **On Success:** Updates the payments record with the found member_id and product_id, and sets status='Matched'. Then, it triggers the Points Calculator.
- **On Failure:** Leaves the payment in Suspense for manual review in the Admin panel.

Why: This is the heart of the "Receipt & Invoice Tracking" and automated points allocation.

4. Points Calculator Function

Triggered by: A successful payment match.

Logic:

- **Input:** payment_id
- Fetches the amount_paid and product_id from the payments table.
- Fetches the points_calculation_ratio for the product from the products table.

- **Calculation:** points_earned = amount_paid / points_calculation_ratio (rounded down).

- **Ledger Update:** Inserts a new record into the loyalty_ledger with the points_earned and updates the cumulative_balance.

- **First Purchase Check:** If this is the member's first successful payment AND they have a referred_by_member_id, it triggers the Referral Bonus Awarder.

Why: To accurately convert money spent into loyalty points based on product-specific rules.

5. Referral Bonus Awarder Function

Triggered by: The Points Calculator upon a member's first successful payment.

Logic:

Input: new_member_id, referrer_member_id

- Defines a fixed bonus amount (e.g., 500 points).
- Creates two transactions in the **loyalty_ledger**:
- **One for the new member:** points_earned = 500, transaction_type = 'Referral Sign-up Bonus'.
- **One for the referrer:** points_earned = 500, transaction_type = 'Successful Referral Bonus'.

Why: To automatically fulfill the "Points addition through referral" requirement.

6. Tier Upgrade Checker Function

Triggered by: Any change to a member's member_products (e.g., a new policy is added).

Logic:

- Counts the number of active products for the member in the member_products table.
- Applies the tier logic: 1=Bronze, 2=Silver, 3=Gold, 5+=Platinum.
- Updates the tier field in the members table if a change is needed.

Why: To automatically manage the "Customer Tiers Tracking" requirement.

7. Points Redemption Endpoint

POST /api/redeem

Logic:

Input: member_id, reward_id

- Checks the member's current cumulative_balance from the loyalty_ledger.

- Fetches the points_cost and checks is_active for the reward from the rewards_catalog.

If valid, it:

- Creates a new points_redemptions record with status='Requested'.

- Inserts a new record into the loyalty_ledger with points_redeemed set to the cost, updating the cumulative_balance.

Why: This is the core of the "Points Usage (deleting)" functionality.

Phase 2: The Control Room - Admin System

Purpose: To provide the Kenbright team with a powerful web-based dashboard to monitor, manage, and control the entire loyalty program.

Core Admin Modules:

1. Dashboard Overview

- **Why:** To see the health of the program at a glance.
- **Features:** Key metrics widgets - Total Members, Tier Distribution, Monthly Revenue, Pending Redemptions, Payments in Suspense.

2. Member Management

- **Why:** To view, search, filter, and manage member profiles.
- **Features:** View member details, tier, points balance, linked products, assets, and dependents.

3. Payment Reconciliation Interface

- **Why:** To manually resolve payments that the auto-matcher failed to reconcile.
- **Features:** A list of Suspense payments. Admins can manually link them to the correct member and product, triggering the points calculation.

4. Rewards Catalog Management

- **Why:** To manage the "Incentives & Discounts" offered in the program.
- **Features:** Add, edit, and activate/deactivate rewards in the rewards_catalog.

5. Redemption Fulfillment Center

- **Why:** To process user redemption requests.
- **Features:** A list of requested redemptions. Admins can update their status to 'Fulfilled' once the discount is applied or the voucher is issued.

6. Reporting & Analytics

- **Why:** For "Internal Tools for Tracking & Segmenting clients".
- **Features:** Visual charts for Product Performance, Points Activity, Cross-sell Rates, and Partner Performance (using the partner_code).

Phase 3: The Member's Portal - User Frontend

Purpose: To provide members with a transparent and engaging view of their loyalty journey, fostering trust and continued interaction.

Core User Features for V1:

1. Secure Login (using Phone Number + OTP)

- **Why:** A simple and secure authentication method widely used in Kenya.

2. Personal Dashboard

- **Why:** The "DISPLAY ON USER DASHBOARD" requirement.
- **Features:** A clear display of the member's current Tier, Point Balance, and a mini-statement of recent points transactions (from the loyalty_ledger).

3. My Profile & KYC

- **Why:** Allows members to view and update their contact information.

4. My Insurance Portfolio

- **Why:** To show members all their products with Kenbright in one place.
- **Features:** A list of their active policies (from member_products).

5. My Assets & Family

- **Why:** Allows members to enrich their profile for better service and cross-selling opportunities.
- **Features:** Forms to add/remove Vehicles, Homes, Spouses, and Children.

6. Rewards Store

- **Why:** The user-facing side of the "Points Usage" requirement.
- **Features:** A page to browse the active rewards_catalog and redeem points.

7. Referral Hub

- **Why:** To empower members to participate in the referral program.
- **Features:** A clear display of their personal referral_code and a simple way to share it (e.g., via WhatsApp, SMS).

Phase 4: Integration, Testing & Deployment

1. Payment Gateway Integration: Connect the system to Safaricom's Daraja API (or other payment processors) to automatically pull payment records into the payments table.

2. SMS/Email Gateway Integration: Integrate a service like Africa's Talking to send automated "Payment Received & Points Added" and "Welcome" notifications.

3. Rigorous Testing:

- **Unit Testing:** Test each API endpoint and calculation function in isolation.
- **Integration Testing:** Test the entire flow from payment to points allocation to tier upgrade.
- **User Acceptance Testing (UAT):** Have a select group of staff and potential users test the complete system.

4. Deployment: Host the backend API on a cloud service (e.g., AWS, DigitalOcean). The Admin and User frontends are deployed as separate, secure web applications.

Improvement & Creative Suggestions for Future Versions

Note: The following are suggestions for future development beyond the V1.0 scope. They are designed to enhance engagement, analytics, and program value.

AI-Powered Cross-Sell Recommendations (V2.0):

- **Suggestion:** Use the data in assets and dependents to power a simple AI model. The system could then show prompts like: "Members who insure their Toyota also protect their home 60% of the time. Get a quote!"
- **Why:** Transforms data into proactive sales opportunities.

Gamification with Badges & Milestones (V2.0):

- **Suggestion:** Award digital badges for achievements beyond tiers.
- **Examples:** "First Payment," "Family Protector" (for adding 3+ dependents), "Wealth Builder" (for purchasing a pension product), "Loyalty Legend" (for 3 consecutive renewals).
- **Why:** Increases psychological engagement and fun.

Partner Ecosystem API (V2.0):

- **Suggestion:** Provide secured APIs for approved partners. This would allow partners to award Kenbright points for their own products/services, significantly expanding the program's reach and value.
- **Why:** Turns the loyalty program into a broader ecosystem.

"Points Earning Potential" Calculator (V1.1):

- **Suggestion:** On product information pages, embed a small calculator. "Buy this KES 15,000 product and earn X points!"
- **Why:** Makes the value of the loyalty program tangible at the point of sale.

Advanced Segmentation & Marketing Tool (V2.0):

- **Suggestion:** Build a tool in the Admin panel that allows marketing to create segments (e.g., "All Gold members with a car but no home insurance") and target them with specific communications or offers.
- **Why:** Drives highly targeted and effective marketing campaigns.

Cross-Reference: Plan vs. Payment Workflow

Payment Workflow Step	Plan Implementation
1. Client Pays Premium	Handled externally via Paybills. The plan's payments table is designed to receive these transactions.
2. System Reconciles Payment	Match. The plan's Payment Reconciliation Engine is a direct implementation of the "Auto Match" and "Manual Match" process described.
3. Points Calculation	Match.
• Four Linked Tables	The plan's 9-table schema includes and expands upon the four core tables (Client, Product, Payment, Loyalty Ledger) with identical linking logic.
• Client Table	Implemented as the members table, including the tier field.
• Product Table	Implemented as the products table, storing the points_calculation_ratio.
• Payment Table	Implemented as the payments table, with status (Matched/Suspense).
• Loyalty Ledger	Implemented as the loyalty_ledger table, with all specified fields and the Cumulative Points logic.
• Calculator Logic	Perfect Match. The plan's Points Calculator Function uses the exact formula: points_earned = amount_paid / points_ratio.
4. Notification	Included in Phase 4 (Integration) via an SMS/Email gateway, sending the exact style of message described.

Cross-Reference: Plan vs. V1.0 Functionality

V1.0 Functionality Requirement	Plan Implementation
1. User Login	Core feature in both User Frontend and Admin System.
2. KYC Storage	Core of the members table and the registration endpoint.
3. Customer Tiers Tracking	tier field in members table + Tier Upgrade Checker Function.
4. Points Tracking (adding)	
• Receipt & Invoice Tracking	payments table + Reconciliation Engine.
• Informing clients on Points	SMS/Email integration in Phase 4.
• Points via referral	Referral Program with referral_code and Referral Bonus Awarder Function.
5. Points Usage (deleting)	
• Usage through product purchase	Rewards & Redemption System (rewards_catalog, points_redemptions tables + Redeem Points Function).
6. Incentives & Discounts	Implemented via the rewards_catalog which defines these incentives.
• Identification with partners	Implemented via the partner_code field in the payments table.
7. Referral Program	Fully implemented as a core V1 feature with codes and bonus points.
8. Internal Tools for Tracking & Segmenting	The entire Admin System is this tool, with a specific Reporting & Analytics module.
• AI Tools	Correctly scoped as a Future Suggestion for V2.0.

Architecture Component Details

1. Frontend Applications Layer

- Admin Dashboard: For Kenbright staff to manage members, payments, rewards
- User Portal: For members to view points, tiers, redeem rewards
- Mobile App: Mobile-first experience for members

2. API Gateway Layer

- Single entry point for all frontend applications
- Handles authentication, rate limiting, request routing
- Provides API versioning and security

3. Backend Services Layer (Microservices Architecture)

- Auth Service: Handles user/admin login, JWT tokens, OTP verification
- Member Service: Manages KYC data, profiles, assets, dependents
- Payment Service: Processes payments, reconciliation, suspense management
- Loyalty Service: Core points and tier management
- Product Service: Insurance product catalog and member-product relationships
- Notification Service: SMS/email alerts for payments and points

4. Core Business Engine (The Brain)

- Payment Reconciliation Engine: Auto-matches payments using unique references
- Points Calculator: Applies product-specific points ratios (1/100 or 1/1000)
- Tier Management Engine: Automatically upgrades members based on product count
- Referral Engine: Manages referral codes and bonus points
- Redemption Engine: Processes points spending and reward fulfillment

5. Database Layer (PostgreSQL)

- 9 core tables with precise relationships
- Foreign key constraints for data integrity
- JSON fields for flexible asset/dependent data storage
- Append-only ledger for audit trail

6. External Integrations

- Safaricom Daraja API: For M-PESA payments and auto-reconciliation
- SMS/Email Gateways: For customer notifications
- Partner APIs: Future expansion for partner rewards
- Analytics Tools: For business intelligence and reporting

Key Data Flows

- **Payment → Points Flow:**
  MPESA Payment → Payment Service → Reconciliation Engine →
  Points Calculator → Loyalty Ledger → SMS Notification

- **Member Registration Flow:**
  User Signup → Auth Service → Member Service →
  Tier Engine → Referral Code Generation

- **Redemption Flow:**
  Reward Selection → Redemption Engine →
  Points Deduction → Admin Fulfillment

This architecture ensures:

- **Scalability:** Microservices can scale independently

- **Maintainability:** Clear separation of concerns

- **Reliability:** Redundant services and failover mechanisms

- **Security:** API gateway with centralized auth

- **Flexibility:** Easy to add new products or features

Backend & Core Engine (Python-Centric)

- Python 3.11+
  - ├── FastAPI (Modern, high-performance web framework)
  - ├── SQLAlchemy (ORM for database operations)
  - ├── Alembic (Database migrations)
  - ├── Pydantic (Data validation and settings management)
  - ├── Celery (Background tasks for payment processing)
  - ├── Redis (Message broker and cache)
  - └── Pandas (Data analysis for reporting)

Database

PostgreSQL 14+
- Robust relational database
- JSON support for flexible asset/dependent data
- Excellent for financial transactions

Frontend Applications

React 18 + TypeScript
- Admin Dashboard
- User Portal
- Vite (Build tool)

Mobile App

React Native
├── Cross-platform mobile app
└── Reuses React knowledge

Infrastructure & DevOps

Docker & Docker Compose
├── Containerization
Nginx
├── Reverse proxy & load balancer
AWS/DigitalOcean
├── Cloud hosting
GitHub Actions
├── CI/CD pipelines

External Integrations

Safaricom Daraja API
├── M-PESA payments
Africa's Talking
├── SMS notifications
SendGrid/Mailgun
├── Email services

FastAPI Backend Structure

```
kenbright360/
├── app/
│   ├── main.py                # FastAPI application entry point
│   ├── core/                  # Core configurations
│   │   ├── config.py          # Settings management
│   │   ├── database.py        # Database connection
│   │   └── security.py        # Authentication utilities
│   ├── models/                # SQLAlchemy models
│   │   ├── member.py
│   │   ├── product.py
│   │   ├── payment.py
│   │   └── loyalty.py
│   ├── schemas/               # Pydantic models for API
│   │   ├── member.py
│   │   ├── payment.py
│   │   └── loyalty.py
│   ├── api/                   # API routes
│   │   ├── endpoints/
│   │   │   ├── auth.py
│   │   │   ├── members.py
│   │   │   ├── payments.py
│   │   │   └── loyalty.py
│   │   └── dependencies.py    # Dependency injection
│   ├── services/              # Business logic
│   │   ├── payment_service.py
│   │   ├── loyalty_service.py
│   │   ├── tier_service.py
│   │   └── referral_service.py
│   ├── workers/               # Celery background tasks
│   │   ├── payment_worker.py
│   │   └── notification_worker.py
│   └── utils/                 # Utilities
│       ├── payment_utils.py
│       ├── calculation_engine.py
│       └── notification.py
├── tests/                     # pytest tests
├── requirements.txt
└── Dockerfile
```